

```

#include <iostream>
#include <string>

using namespace std;

const int NUM_REGS = 16;
const int NUM_MEM = 16;
struct DecodedInstruction {
    unsigned int raw;
    unsigned int opcode;
    unsigned int rs;
    unsigned int rt;
    unsigned int rd;
    unsigned int shamt;
    unsigned int funct;
    unsigned int imm;
    char format;
};

// Print out all the registers and their values
void printRegs(const int regs[NUM_REGS]){
    cout << "Registers: " << endl;
    for (int i = 0; i < NUM_REGS; i++){
        cout << "R" << i << ": " << regs[i] << endl;
    }
    cout << endl;
}

// Print out all memory locations and thier values
void printMem(const int mem[NUM_MEM]){
    cout << "Memory: " << endl;
    for (int i = 0; i < NUM_MEM; i++){
        cout << "M[" << i << "]: " << mem[i] << endl;
    }
    cout << endl;
}

//Check a letter is a valid
bool isHexChar(char c){
    if(c>='0' && c<='9') return true;
    if(c>='A' && c<='F') return true;
    if(c>='a' && c<='f') return true;
    return false;
}

```

```
}
```

```
//Convert Hex Charectors to values using their ascii values
```

```
int hexCharToValue(char c) {  
    if (c >= '0' && c <= '9') {  
        return c - '0';  
    }  
    else if (c >= 'A' && c <= 'F') {  
        return 10 + (c - 'A');  
    }  
    else if (c >= 'a' && c <= 'f') {  
        return 10 + (c - 'a');  
    }  
    else {  
        return 0;  
    }  
}
```

```
//convert hex string to values. I will use this value to perform task
```

```
unsigned int parseHexInstruction(const string& hex) {  
    unsigned int value = 0;  
  
    for (char c : hex) {  
        int digit = hexCharToValue(c); // 0–15  
  
        value = (value << 4);  
        // scoots everything to the left by 4 bits and puts 0's in those 4 bits  
        value |= digit;  
        // performs an or operation on value and digits to add digit to the end of value  
    }  
    return value;  
}
```

```
//convert 32 bit number into binary string so we can print it
```

```
string toBinaryString(unsigned int value) {  
    string result;  
  
    for (int i = 31; i >= 0; i--) {  
        // i use this to look at each postion  
        unsigned int mask = 1u << i;  
  
        // conditional to see if each bit is set  
        if (value & mask) {  
            result.push_back('1');  
        }  
    }  
}
```

```

    } else {
        result.push_back('0');
    }

    // add a space intbeen every 4 bits
    if (i % 4 == 0 && i != 0) {
        result.push_back(' ');
    }
}

return result;
}

```

```

//get each part of the hex instruction
DecodedInstruction decodeInstruction(unsigned int instr) {
    DecodedInstruction d{};
    d.raw = instr;

    d.opcode = (instr >> 26) & 0x3F;
    d.rs     = (instr >> 21) & 0x1F;
    d.rt     = (instr >> 16) & 0x1F;
    d.rd     = (instr >> 11) & 0x1F;
    d.shamt  = (instr >> 6) & 0x1F;
    d.funct  = instr      & 0x3F;
    d.imm    = instr      & 0xFFFF;

    if (d.opcode == 0) {
        d.format = 'R';
    } else {
        d.format = 'I';
    }

    return d;
}

```

```

//Print the decoded instruction from the strut
void printDecodedInstruction(const DecodedInstruction& d) {
    cout << "Format: " << d.format << "-type" << endl;
    cout << "Opcode: " << d.opcode << endl;
    cout << "rs: " << d.rs << endl;
    cout << "rt: " << d.rt << endl;

    if (d.format == 'R') {

```

```

    cout << "rd: " << d.rd << endl;
    cout << "shamt: " << d.shamt << endl;
    cout << "funct: " << d.funct << endl;
} else { // I-type
    cout << "Immediate (imm): " << d.imm << endl;
}

cout << "Assembly: ";
if (d.format == 'R') {
    switch (d.funct) {
        case 0x20: // add
            cout << "add $" << d.rd << ", $" << d.rs << ", $" << d.rt;
            break;
        case 0x22: // sub
            cout << "sub $" << d.rd << ", $" << d.rs << ", $" << d.rt;
            break;
        case 0x24: // and
            cout << "and $" << d.rd << ", $" << d.rs << ", $" << d.rt;
            break;
        case 0x25: // or
            cout << "or $" << d.rd << ", $" << d.rs << ", $" << d.rt;
            break;
        case 0x26: // xor
            cout << "xor $" << d.rd << ", $" << d.rs << ", $" << d.rt;
            break;
        default:
            cout << "Unknown R-type instruction (funct = " << d.funct << ")";
            break;
    }
} else { // I-type
    // sign-extend imm for display (same as executeInstruction)
    int immSigned = static_cast<int16_t>(d.imm);

    switch (d.opcode) {
        case 0x08: // addi
            cout << "addi $" << d.rt << ", $" << d.rs << ", " << immSigned;
            break;
        case 0x23: // lw
            cout << "lw $" << d.rt << ", " << immSigned << "(" << d.rs << ")";
            break;
        case 0x2B: // sw
            cout << "sw $" << d.rt << ", " << immSigned << "(" << d.rs << ")";
            break;
        default:

```

```

        cout << "Unknown I-type instruction (opcode = " << d.opcode << ")";
        break;
    }
}
cout << "\n\n";
}

```

//execut the hex isntruction

```

void executeInstruction(const DecodedInstruction& d, int regs[NUM_REGS], int
mem[NUM_MEM]) {

```

```

    //make sure register index is valid
    auto validReg = [](unsigned int r) {
        return r < NUM_REGS;
    };

```

```

    if (d.format == 'R') {
        switch (d.funct) {
            // add rd = rs + rt
            case 0x20:
                if (validReg(d.rs) && validReg(d.rt) && validReg(d.rd)) {
                    regs[d.rd] = regs[d.rs] + regs[d.rt];
                }
                break;

```

```

            // sub rd = rs - rt
            case 0x22:
                if (validReg(d.rs) && validReg(d.rt) && validReg(d.rd)) {
                    regs[d.rd] = regs[d.rs] - regs[d.rt];
                }
                break;

```

```

            // AND rd = rs & rt
            case 0x24:
                if (validReg(d.rs) && validReg(d.rt) && validReg(d.rd)) {
                    regs[d.rd] = regs[d.rs] & regs[d.rt];
                }
                break;

```

```

            // OR rd = rs | rt
            case 0x25:
                if (validReg(d.rs) && validReg(d.rt) && validReg(d.rd)) {
                    regs[d.rd] = regs[d.rs] | regs[d.rt];

```

```

    }
    break;

// XOR rd = rs ^ rt
case 0x26:
    if (validReg(d.rs) && validReg(d.rt) && validReg(d.rd)) {
        regs[d.rd] = regs[d.rs] ^ regs[d.rt];
    }
    break;

default:
    cout << "Unknown R-type funct: " << d.funct << endl;
    break;
}
} else { // I-type
    switch (d.opcode) {
        // addi rt = rs + imm
        case 0x08: {
            if (validReg(d.rs) && validReg(d.rt)) {
                int imm = static_cast<int16_t>(d.imm); // sign-extend
                regs[d.rt] = regs[d.rs] + imm;
            }
            break;
        }

        // lw rt, imm(rs)
        case 0x23: {
            if (validReg(d.rs) && validReg(d.rt)) {
                int imm = static_cast<int16_t>(d.imm); // sign-extend
                int addr = regs[d.rs] + imm;
                int index = addr / 4; // word addressing

                if (index >= 0 && index < NUM_MEM) {
                    regs[d.rt] = mem[index];
                } else {
                    cout << "lw: memory address out of range (index = "
                        << index << ")" << endl;
                }
            }
            break;
        }

        // sw rt, imm(rs)
        case 0x2B: {

```

```

        if (validReg(d.rs) && validReg(d.rt)) {
            int imm = static_cast<int16_t>(d.imm); // sign-extend
            int addr = regs[d.rs] + imm;
            int index = addr / 4;

            if (index >= 0 && index < NUM_MEM) {
                mem[addr] = regs[d.rt];
            } else {
                cout << "sw: memory address out of range (index = "
                    << index << ")" << endl;
            }
        }
        break;
    }
}

default:
    cout << "Unknown I-type opcode: " << d.opcode << endl;
    break;
}
}
}

```

//Check if user input is a valid hex instruction

```

bool isValidHexInstruction(const string& s){
    if (s.size() != 8) return false;

    for (char c : s){
        if(!isHexChar(c)) {
            return false;
        }
    }
    return true;
}

```

//get the hex instruction from user and use previous functions to validate instruction

```

string getHexInstructionFromUser(){
    string instr;

    while(true) {
        cout << "Please enter an 8-digit hexadecimal instruction (or q to quit): ";
        cin >> instr;
    }
}

```

```

    if (instr == "q" || instr == "Q") return instr;
    if (isValidHexInstruction(instr)){
        return instr;
    }
    else {
        cout << "Invalid input. Please re-enter an actual hex instruction" << endl;
    }
}
}

```

```

int main() {
    int regs[NUM_REGS] = {0000};
    int mem[NUM_MEM] = {0000};

    for (int i = 0; i < NUM_REGS; i++) {
        regs[i] = i;
        mem[i] = i;
    }

    while (true) {
        printRegs(regs);
        printMem(mem);

        string hexInstr = getHexInstructionFromUser();

        if (hexInstr == "q" || hexInstr == "Q") {
            cout << "Exiting the program" << endl;
            break;
        }

        unsigned int instruction = parseHexInstruction(hexInstr);
        cout << "You entered instruction: " << hexInstr << endl;
        cout << "Instruction as unsigned int: " << instruction << endl;

        string binary = toBinaryString(instruction);
        cout << "Binary representation: " << binary << endl;

        DecodedInstruction decoded = decodeInstruction(instruction);
        printDecodedInstruction(decoded);

        executeInstruction(decoded, regs, mem);
    }
}

```



```
}  
  
return 0;  
}
```

%7D%0D%0A++++return+value%3B%0D%0A%7D%0D%0A%0D%0A%2F%2Fconvert+32+bit
+number+into+binary+string+so+we+can+print+it%0D%0A++++string+toBinaryString(unsigned
+int+value)+%7B%0D%0A++++string+result%3B%0D%0A++++%0D%0A++++for+(i
nt+i+%3D+31%3B+i+%3E%3D+0%3B+i--)+%7B%0D%0A++++%2F%2F+i+use+this
+to+look+at+each+postion%0D%0A++++unsigned+int+mask+%3D+1u+%3C%3C+i
%3B%0D%0A++++%0D%0A++++%2F%2F+conditional+to+see+if+each+
bit+is+set%0D%0A++++if+(value+%26+mask)+%7B%0D%0A++++re
sult.push_back(%271%27)%3B%0D%0A++++%7D+else+%7B%0D%0A++++
++++result.push_back(%270%27)%3B%0D%0A++++%7D%0D%0A++++
++%0D%0A++++%2F%2F+add+a+space+intbeen+every+4+bits%0D%0A++++
++++if+(i+%25+4+%3D%3D+0+%26%26+i!%3D+0)+%7B%0D%0A++++result
_push_back(%27+%27)%3B%0D%0A++++%7D%0D%0A++++%7D%0D%0A++
++++%0D%0A++++return+result%3B%0D%0A%7D%0D%0A%0D%0A%2F%2Fget+each+p
art+of+the+hex+instruction%0D%0ADecodedInstruction+decodeInstruction(unsigned+int+instr)
+%7B%0D%0A++++DecodedInstruction+d%7B%7D%3B%0D%0A++++d.raw+%3D+instr%3B
%0D%0A++++%0D%0A++++d.opcode+%3D+(instr+%3E%3E+26)+%26+0x3F%3B++%0D%0
A++++d.rs++++%3D+(instr+%3E%3E+21)+%26+0x1F%3B++%0D%0A++++d.rt++++%3D+(i
nstr+%3E%3E+16)+%26+0x1F%3B++%0D%0A++++d.rd++++%3D+(instr+%3E%3E+11)+%2
6+0x1F%3B++%0D%0A++++d.shamt+%3D+(instr+%3E%3E+6)+%26+0x1F%3B++%0D%0
A++++d.funct+%3D+instr++++%26+0x3F%3B++%0D%0A++++d.imm++++%3D+instr
++++%26+0xFFFF%3B%0D%0A++++%0D%0A++++%0D%0A++++if+(d.opcode+%3D%3
D+0)+%7B%0D%0A++++d.format+%3D+%27R%27%3B%0D%0A++++%7D+else+%7B%
0D%0A++++d.format+%3D+%27I%27%3B%0D%0A++++%7D%0D%0A++++%0D%0A++
++return+d%3B%0D%0A%7D%0D%0A%0D%0A%2F%2FPrint+the+decoded+instruction+from
+the+strut%0D%0Avoid+printDecodedInstruction(const+DecodedInstruction%26+d)+%7B%0D
%0A++++cout+%3C%3C+%22Format%3A+%22+%3C%3C+d.format+%3C%3C+%22-type%2
2+%3C%3C+endl%3B%0D%0A++++cout+%3C%3C+%22Opcode%3A+%22+%3C%3C+d.opc
ode+%3C%3C+endl%3B%0D%0A++++cout+%3C%3C+%22rs%3A+%22+%3C%3C+d.rs+%3
C%3C+endl%3B%0D%0A++++cout+%3C%3C+%22rt%3A+%22+%3C%3C+d.rt+%3C%3C+en
dl%3B%0D%0A%0D%0A++++if+(d.format+%3D%3D+%27R%27)+%7B%0D%0A++++co
ut+%3C%3C+%22rd%3A+%22+%3C%3C+d.rd+%3C%3C+endl%3B%0D%0A++++cout+
%3C%3C+%22shamt%3A+%22+%3C%3C+d.shamt+%3C%3C+endl%3B%0D%0A++++c
out+%3C%3C+%22funct%3A+%22+%3C%3C+d.funct+%3C%3C+endl%3B%0D%0A++++%7D
+else+%7B+%2F%2F+I-type%0D%0A++++cout+%3C%3C+%22Immediate+(imm)%3A+
%22+%3C%3C+d.imm+%3C%3C+endl%3B%0D%0A++++%7D%0D%0A%0D%0A++++cout+
%3C%3C+%22Assembly%3A+%22%3B%0D%0A++++if+(d.format+%3D%3D+%27R%27)+%7
B%0D%0A++++switch+(d.funct)+%7B%0D%0A++++case+0x20%3A+%2F%2F
+add%0D%0A++++cout+%3C%3C+%22add+%24%22+%3C%3C+d.rd+%3C%
3C+%22%2C+%24%22+%3C%3C+d.rs+%3C%3C+%22%2C+%24%22+%3C%3C+d.rt%3B%0
D%0A++++break%3B%0D%0A++++case+0x22%3A+%2F%2F+sub
%0D%0A++++cout+%3C%3C+%22sub+%24%22+%3C%3C+d.rd+%3C%3C+
%22%2C+%24%22+%3C%3C+d.rs+%3C%3C+%22%2C+%24%22+%3C%3C+d.rt%3B%0D%
0A++++break%3B%0D%0A++++case+0x24%3A+%2F%2F+and%0D
%0A++++cout+%3C%3C+%22and+%24%22+%3C%3C+d.rd+%3C%3C+%22

%2C+%24%22+%3C%3C+d.rs+%3C%3C+%22%2C+%24%22+%3C%3C+d.rt%3B%0D%0A+
+++++++break%3B%0D%0A+++++++case+0x25%3A+%2F%2F+or%0D%0A
+++++++cout+%3C%3C+%22or+%24%22+%3C%3C+d.rd+%3C%3C+%22%2C+
%24%22+%3C%3C+d.rs+%3C%3C+%22%2C+%24%22+%3C%3C+d.rt%3B%0D%0A+++++
+++++++break%3B%0D%0A+++++++case+0x26%3A+%2F%2F+xor%0D%0A++++
+++++++cout+%3C%3C+%22xor+%24%22+%3C%3C+d.rd+%3C%3C+%22%2C+%24
%22+%3C%3C+d.rs+%3C%3C+%22%2C+%24%22+%3C%3C+d.rt%3B%0D%0A+++++++
+++++++break%3B%0D%0A+++++++default%3A%0D%0A+++++++cout+%
3C%3C+%22Unknown+R-type+instruction+(funct+%3D+%22+%3C%3C+d.funct+%3C%3C+%
22)%22%3B%0D%0A+++++++break%3B%0D%0A+++++++%7D%0D%0A++++%
7D+else+%7B+%2F%2F+l-type%0D%0A+++++++%2F%2F+sign-extend+imm+for+display+(s
ame+as+executeInstruction)%0D%0A+++++++int+immSigned+%3D+static_cast%3Cint16_t%
3E(d.imm)%3B%0D%0A%0D%0A+++++++switch+(d.opcode)+%7B%0D%0A+++++++
case+0x08%3A+%2F%2F+addi%0D%0A+++++++cout+%3C%3C+%22addi+%24%
22+%3C%3C+d.rt+%3C%3C+%22%2C+%24%22+%3C%3C+d.rs+%3C%3C+%22%2C+%22+
%3C%3C+immSigned%3B%0D%0A+++++++break%3B%0D%0A+++++++c
ase+0x23%3A+%2F%2F+lw%0D%0A+++++++cout+%3C%3C+%22lw+%24%22+
%3C%3C+d.rt+%3C%3C+%22%2C+%22+%3C%3C+immSigned+%3C%3C+%22(%24%22+%
3C%3C+d.rs+%3C%3C+%22)%22%3B%0D%0A+++++++break%3B%0D%0A++++
+++++++case+0x2B%3A+%2F%2F+sw%0D%0A+++++++cout+%3C%3C+%22sw
+%24%22+%3C%3C+d.rt+%3C%3C+%22%2C+%22+%3C%3C+immSigned+%3C%3C+%22(
%24%22+%3C%3C+d.rs+%3C%3C+%22)%22%3B%0D%0A+++++++break%3B%
0D%0A+++++++default%3A%0D%0A+++++++cout+%3C%3C+%22Unknow
n+l-type+instruction+(opcode+%3D+%22+%3C%3C+d.opcode+%3C%3C+%22)%22%3B%0D
%0A+++++++break%3B%0D%0A+++++++%7D%0D%0A++++%7D%0D%0A+++
+cout+%3C%3C+%22%5Cn%5Cn%22%3B%0D%0A%7D%0D%0A%0D%0A%0D%0A%2F%2
Fexecute+the+hex+is+intrusion%0D%0Aavoid+executeInstruction(const+DecodedInstruction%26+d
%2C+int+regs%5BNUM_REGS%5D%2C+int+mem%5BNUM_MEM%5D)+%7B%0D%0A++++
%0D%0A++++%2F%2Fmake+sure+register+index+is+valid%0D%0A++++auto+validReg+%3D
+%5B%5D(unsigned+int+r)+%7B%0D%0A+++++++return+r+%3C+NUM_REGS%3B%0D%0
A++++%7D%3B%0D%0A%0D%0A++++if+(d.format+%3D%3D+%27R%27)+%7B%0D%0A+++
++++switch+(d.funct)+%7B%0D%0A+++++++%2F%2F+add+rd+%3D+rs+%2B+rt%0D
%0A+++++++case+0x20%3A+%0D%0A+++++++if+(validReg(d.rs)+%26%26
+validReg(d.rt)+%26%26+validReg(d.rd))+%7B%0D%0A+++++++regs%5Bd.r
d%5D+%3D+regs%5Bd.rs%5D+%2B+regs%5Bd.rt%5D%3B%0D%0A+++++++%7
D%0D%0A+++++++break%3B%0D%0A+++++++%0D%0A+++++++
+++%2F%2F+sub+rd+%3D+rs+-+rt%0D%0A+++++++case+0x22%3A+%0D%0A+++++
+++++++if+(validReg(d.rs)+%26%26+validReg(d.rt)+%26%26+validReg(d.rd))+%7B%0D%0
A+++++++regs%5Bd.rd%5D+%3D+regs%5Bd.rs%5D+-+regs%5Bd.rt%5D%3
B%0D%0A+++++++%7D%0D%0A+++++++break%3B%0D%0A%0D%
0A+++++++%2F%2F+AND+rd+%3D+rs+%26+rt%0D%0A+++++++case+0x24%3
A%0D%0A+++++++if+(validReg(d.rs)+%26%26+validReg(d.rt)+%26%26+validReg(
d.rd))+%7B%0D%0A+++++++regs%5Bd.rd%5D+%3D+regs%5Bd.rs%5D+%2
6+regs%5Bd.rt%5D%3B%0D%0A+++++++%7D%0D%0A+++++++brea

k%3B%0D%0A%0D%0A++++++%2F%2F+OR+rd+%3D+rs+%7C+rt%0D%0A++++++
+++++case+0x25%3A%0D%0A++++++if+(validReg(d.rs)+%26%26+validReg(d.rt)
+%26%26+validReg(d.rd))+%7B%0D%0A++++++regs%5Bd.rd%5D+%3D+r
egs%5Bd.rs%5D+%7C+regs%5Bd.rt%5D%3B%0D%0A++++++%7D%0D%0A+++
+++++break%3B%0D%0A%0D%0A++++++%2F%2F+XOR+rd+%3D+rs+%5
E+rt%0D%0A++++++case+0x26%3A%0D%0A++++++if+(validReg(d.rs)+
%26%26+validReg(d.rt)+%26%26+validReg(d.rd))+%7B%0D%0A++++++reg
s%5Bd.rd%5D+%3D+regs%5Bd.rs%5D+%5E+regs%5Bd.rt%5D%3B%0D%0A++++++
++++%7D%0D%0A++++++break%3B%0D%0A%0D%0A++++++default%
3A%0D%0A++++++cout+%3C%3C+%22Unknown+R-type+funct%3A+%22+%3C
%3C+d.funct+%3C%3C+endl%3B%0D%0A++++++break%3B%0D%0A++++++
%7D%0D%0A++++%7D+else+%7B+%2F%2F+I-type%0D%0A++++++switch+(d.opcode)+%
7B%0D%0A++++++%2F%2F+addi+rt+%3D+rs+%2B+imm%0D%0A++++++cas
e+0x08%3A+%7B+%0D%0A++++++if+(validReg(d.rs)+%26%26+validReg(d.rt))+
%7B%0D%0A++++++int+imm+%3D+static_cast%3Cint16_t%3E(d.imm)%3B
+%2F%2F+sign-extend%0D%0A++++++regs%5Bd.rt%5D+%3D+regs%5Bd.
rs%5D+%2B+imm%3B%0D%0A++++++%7D%0D%0A++++++break
%3B%0D%0A++++++%7D%0D%0A%0D%0A++++++%2F%2F+lw+rt%2C+im
m(rs)%0D%0A++++++case+0x23%3A+%7B%0D%0A++++++if+(validReg(
d.rs)+%26%26+validReg(d.rt))+%7B%0D%0A++++++int+imm+%3D+static_c
ast%3Cint16_t%3E(d.imm)%3B+%2F%2F+sign-extend%0D%0A++++++int+
addr+%3D+regs%5Bd.rs%5D+%2B+imm%3B%0D%0A++++++int+index+%3
D+addr+%2F+4%3B+%2F%2F+word+addressing%0D%0A%0D%0A++++++
if+(index+%3E%3D+0+%26%26+index+%3C+NUM_MEM)+%7B%0D%0A++++++
++++++regs%5Bd.rt%5D+%3D+mem%5Bindex%5D%3B%0D%0A++++++
+%7D+else+%7B%0D%0A++++++cout+%3C%3C+%22lw%3A+memor
y+address+out+of+range+(index+%3D+%22+%0D%0A++++++%
3C%3C+index+%3C%3C+%22)%22+%3C%3C+endl%3B%0D%0A++++++
%7D%0D%0A++++++%7D%0D%0A++++++break%3B%0D%0A+++
++++++%7D%0D%0A%0D%0A++++++%2F%2F+sw+rt%2C+imm(rs)%0D%0A+++
++++++case+0x2B%3A+%7B%0D%0A++++++if+(validReg(d.rs)+%26%26+va
lidReg(d.rt))+%7B%0D%0A++++++int+imm+%3D+static_cast%3Cint16_t%3
E(d.imm)%3B+%2F%2F+sign-extend%0D%0A++++++int+addr+%3D+regs%
5Bd.rs%5D+%2B+imm%3B%0D%0A++++++int+index+%3D+addr+%2F+4%
3B%0D%0A%0D%0A++++++if+(index+%3E%3D+0+%26%26+index+%3C+
NUM_MEM)+%7B%0D%0A++++++mem%5Baddr%5D+%3D+regs%5B
d.rt%5D%3B%0D%0A++++++%7D+else+%7B%0D%0A++++++
++++++cout+%3C%3C+%22sw%3A+memory+address+out+of+range+(index+%3D+%22+
%0D%0A++++++%3C%3C+index+%3C%3C+%22)%22+%3C%3
C+endl%3B%0D%0A++++++%7D%0D%0A++++++%7D%0D%
0A++++++break%3B%0D%0A++++++%7D%0D%0A%0D%0A++++++
+++default%3A%0D%0A++++++cout+%3C%3C+%22Unknown+I-type+opcode%3
A+%22+%3C%3C+d.opcode+%3C%3C+endl%3B%0D%0A++++++break%3B%0
D%0A++++++%7D%0D%0A++++%7D%0D%0A%7D%0D%0A%0D%0A%0D%0A%0D%0A

```
%2F%2FCheck+if+user+input+is+a+valid+hex+instructiom%0D%0Abool+isValidHexInstruction(  
const+string%26+s)%7B%0D%0A++++if+(s.size()+!%3D+8)+return+false%3B%0D%0A++++%  
0D%0A++++for+(char+c+%3A+s)%7B%0D%0A+++++if+(!isHexChar(c))+%7B%0D%0A+++  
+++++return+false%3B%0D%0A+++++%7D%0D%0A++++%7D%0D%0A++++return+  
true%3B%0D%0A%7D%0D%0A%0D%0A%2F%2Fget+the+hex+instruction+from+user+and+u  
se+prevoius+functions+to+validate+instruction%0D%0Astring+getHexInstructionFromUser()%7  
B%0D%0A++++string+instr%3B%0D%0A++++%0D%0A++++while(true)+%7B%0D%0A+++++  
+++cout+%3C%3C+%22Please+enter+an+8-digit+hexidecimal+instruction+(or+q+to+quit)%3A  
+%22%3B%0D%0A+++++cin+%3E%3E+instr%3B%0D%0A+++++%0D%0A+++++i  
f+(instr+%3D%3D+%22q%22+%7C%7C+instr+%3D%3D+%22Q%22)+return+instr%3B%0D%0  
A+++++if+(isValidHexInstruction(instr))%7B%0D%0A+++++return+instr%3B%0D%0  
A+++++%7D%0D%0A+++++else+%7B%0D%0A+++++cout+%3C%3C+%22I  
nvalid+input.+Please+re-enter+an+actual+hex+instruction%22+%3C%3C+endl%3B%0D%0A+  
+++++%7D%0D%0A++++%7D%0D%0A%7D%0D%0A+++++%0D%0A++++%0D%0A%  
0D%0Aint+main()+%7B%0D%0A++++int+regs%5BNUM_REGS%5D+%3D+%7B0000%7D%3  
B%0D%0A++++int+mem%5BNUM_MEM%5D+%3D+%7B0000%7D%3B%0D%0A++++%0D%  
0A++++for+(int+i+%3D+0%3B+i+%3C+NUM_REGS%3B+i%2B%2B)+%7B%0D%0A++++regs  
%5Bi%5D+%3D+i%3B%0D%0A++++mem%5Bi%5D+%3D+i%3B%0D%0A++++%7D%0D%0  
A%0D%0A++++while+(true)+%7B%0D%0A+++++printRegs(regs)%3B%0D%0A+++++  
printMem(mem)%3B%0D%0A%0D%0A+++++string+hexInstr+%3D+getHexInstructionFrom  
User()%3B%0D%0A%0D%0A+++++if+(hexInstr+%3D%3D+%22q%22+%7C%7C+hexInstr  
+%3D%3D+%22Q%22)+%7B%0D%0A+++++cout+%3C%3C+%22Exiting+the+progra  
m%22+%3C%3C+endl%3B%0D%0A+++++break%3B%0D%0A+++++%7D%0D%0  
A%0D%0A+++++unsigned+int+instruction+%3D+parseHexInstruction(hexInstr)%3B%0D  
%0A+++++cout+%3C%3C+%22You+entered+instruction%3A+%22+%3C%3C+hexInstr+%  
3C%3C+endl%3B%0D%0A+++++cout+%3C%3C+%22Instruction+as+unsigned+int%3A+%  
22+%3C%3C+instruction+%3C%3C+endl%3B%0D%0A%0D%0A+++++string+binary+%3D  
+toBinaryString(instruction)%3B%0D%0A+++++cout+%3C%3C+%22Binary+representation  
%3A+%22+%3C%3C+binary+%3C%3C+endl%3B%0D%0A%0D%0A+++++DecodedInstru  
ction+decoded+%3D+decodeInstruction(instruction)%3B%0D%0A+++++printDecodedInstru  
ction(decoded)%3B%0D%0A%0D%0A+++++executeInstruction(decoded%2C+regs%2C+m  
em)%3B%0D%0A%0D%0A++++%7D%0D%0A%0D%0A++++return+0%3B%0D%0A%7D%0D  
%0A%0D%0A+++++
```